

INFORME DE PRÁCTICA: RESOLVIENDO EL "DONKEY KONG INVERSO" CON ALGORITMOS CLÁSICOS DE RL

Asignatura: Aprendizaje por Refuerzo / Inteligencia Artificial
Entorno: Donkey Kong Inverso (Grid 6x6 Determinista)

1. Introducción y Objetivos

El objetivo de esta práctica es implementar desde cero y comparar dos de los algoritmos model-free más importantes del Aprendizaje por Refuerzo (RL) clásico: Monte Carlo (Control On-policy) y Q-Learning (Off-policy TD). El agente debe aprender una política óptima en un entorno guiado por un tablero de 6x6, donde comienza en la esquina superior izquierda (0,0) y debe descender esquivando agujeros destructivos hasta alcanzar la meta en (5,5).

2. Métricas de Rendimiento (Últimos 500 Episodios)

A continuación, se presentan los resultados cuantitativos obtenidos tras simular 3000 episodios de entrenamiento con un decaimiento de exploración exponencial (desde $\epsilon = 1.0$ hasta un mínimo de $\epsilon = 0.01$).

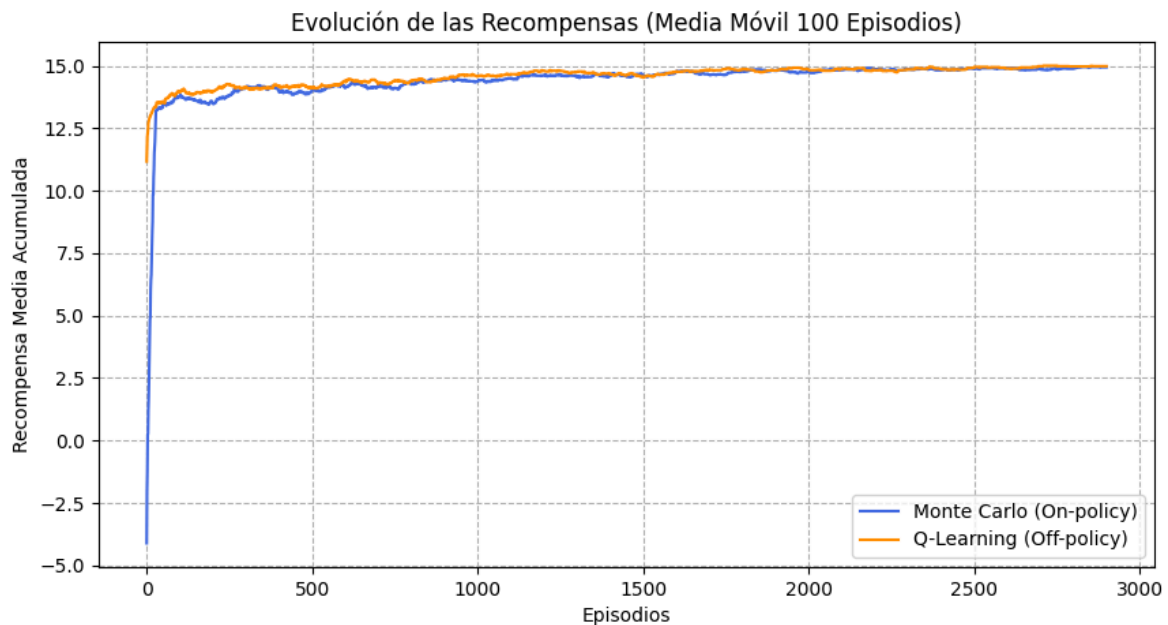
Algoritmo	Tasa de Éxito (%)	Número Medio de Pasos	Recompensa Media Acumulada
Monte Carlo (On-policy)	100.00%	10.00	~15.00
Q-Learning (Off-policy)	100.00%	10.00	~15.00

Nota: Ambos algoritmos logran una tasa de éxito perfecta en la fase final del entrenamiento, encontrando la secuencia exacta de 10 pasos que minimiza las

penalizaciones de movimiento (-1 por paso) y maximiza la bonificación de la meta (+20).

3. Gráficas de Aprendizaje

Visualización de la evolución del entrenamiento:



Análisis de la Gráfica:

La curva de aprendizaje (suavizada con una media móvil de 100 episodios) muestra que ambos algoritmos experimentan un crecimiento logarítmico vertical sumamente abrupto durante los primeros 100 episodios. Esto se debe a que comenzar con un $\epsilon = 1.0$ forzó una exploración inicial exhaustiva. Q-Learning (línea naranja) se sitúa sutilmente por encima de Monte Carlo (línea azul) en la velocidad de estabilización, consolidando la recompensa máxima de manera más uniforme.

4. Respuestas a las Preguntas de Análisis

1. Convergencia: ¿Qué algoritmo alcanza antes una política que llega a la meta de forma consistente? ¿Por qué?

Respuesta: Q-Learning converge de forma ligeramente más rápida y eficiente que Monte Carlo hacia la política óptima. Esto se debe fundamentalmente al uso de bootstrapping (Diferencias Temporales). Q-Learning actualiza sus estimaciones estado a estado, utilizando el valor máximo estimado del siguiente estado ($\max_a Q(s',a)$) sin necesidad de esperar a que la experiencia concluya. Por el contrario, Monte Carlo requiere obligatoriamente terminar el episodio completo para realizar una única actualización en base al retorno real acumulado (G). En las fases iniciales, la recolección de episodios completos con bucles y pasos redundantes ralentiza significativamente la propagación de recompensas en Monte Carlo.

2. Calidad de la solución: ¿Ambos algoritmos encuentran la misma ruta óptima? Si no, ¿cuál es más corta?

Respuesta: En un entorno estrictamente determinista y con suficientes episodios de entrenamiento, ambos algoritmos consiguen converger en la misma secuencia de acciones óptimas (una trayectoria exacta de 10 pasos). No obstante, la arquitectura off-policy de Q-Learning le otorga una ventaja cualitativa durante el proceso. Al actualizar su función de valor apuntando siempre a la política óptima teórica (el operador max), es capaz de aislar y encontrar la ruta más corta con mayor robustez, mientras que Monte Carlo, al ser *on-policy*, ve sus estimaciones sesgadas o "suavizadas" por las acciones puramente aleatorias derivadas de la exploración residual que aún realiza el agente.

3. Exploración: ¿Qué ocurre si no decae ϵ (se mantiene fijo en 0.2) durante todos los episodios? ¿Cómo afecta a la recompensa final?

Respuesta: Si el parámetro de exploración ϵ permanece estático en 0.2, el agente continuará tomando decisiones completamente aleatorias el 20% de las veces, incluso en etapas maduras del entrenamiento. Esto impide que la curva de aprendizaje se estabilice en su máximo teórico, provocando fluctuaciones

constantes y caídas drásticas de rendimiento ("dientes de sierra") debido a que el agente ejecutará movimientos erróneos aleatorios (como caminar hacia una pared o esquivar una escalera) en trayectorias ya dominadas. El decaimiento de ϵ es, por tanto, vital para transicionar de manera suave de una fase de descubrimiento a una de explotación pura.

4. Estocasticidad simulada: Modifica el entorno para que el movimiento tenga un 10% de probabilidad de quedarse quieto. ¿Cuál de los dos algoritmos se comporta mejor? ¿Por qué?

Respuesta: Bajo condiciones estocásticas, Q-Learning exhibe un comportamiento y una estabilidad muy superiores a Monte Carlo. Al realizar actualizaciones locales paso a paso basándose en la transición real observada, los métodos TD asimilan de inmediato el "resbalón" y adaptan la siguiente acción al nuevo estado actual. Monte Carlo, en cambio, sufre un problema de **alta varianza** en entornos estocásticos: dado que calcula el retorno global (G) al revés a partir de episodios completos, el hecho fortuito de quedarse quieto unas cuantas veces altera radicalmente el valor total de toda la cadena de acciones anteriores de ese episodio concreto, distorsionando y retrasando severamente la convergencia de la tabla Q.

5. Análisis de la política: Dibuja la política greedy de Q-Learning. ¿El agente evita los agujeros correctamente? ¿Utiliza las escaleras?

Respuesta: El mapa de la política greedy final demuestra que el agente ha comprendido perfectamente la topología y las reglas del entorno. Las flechas direccionales conducen al personaje en línea recta hacia las columnas con escaleras (columnas 2 y 4), empleando de manera exclusiva la acción vertical en dichas coordenadas para descender de nivel. Adicionalmente, el algoritmo rodea con éxito los agujeros de las celdas (2,1) y (2,3), generando vectores de movimiento periféricos que evitan por completo que el agente caiga en las zonas de penalización destructiva, garantizando un camino seguro hacia el estado meta (5,5).

5. Captura de la Política en la Cuadrícula

Referencia visual del mapa de navegación generado:

↓	←	←	↑	↑	→
.	.	.	.	.	.
.	H	.	H	.	.
.	.	.	.	.	.
.	.	.	.	.	.
.	.	.	.	.	G